

Relatório Assignment #3 – Fundamentos de Segurança Informática

Realizado por:

Gustavo Roberto Teodoro Maques

2023215017 2023211717

1. Descrição das Configurações e Implementação da Arquitetura

A infraestrutura foi montada em VMware com três máquinas virtuais CentOS 9 ligadas pelo LAN Segment *FSI-Network*, isolando o ambiente do exterior. O objetivo do trabalho dividiu-se em duas fases: na Fase 1 foram realizados testes de segurança ao OWASP Juice Shop seguindo o guia OWASP WSTG v4.2; na Fase 2 foi colocado um Web Application Firewall (Apache + ModSecurity + OWASP CRS) em frente da aplicação e os mesmos testes foram repetidos para avaliar a eficácia da proteção.

1.1. Topologia de Rede

A rede interna opera no segmento 192.168.100.0/24 com a seguinte distribuição:

- **VM 1 (client-vm):** 192.168.100.10. Máquina de testes com OWASP ZAP 2.16.0 e Firefox configurado com o proxy do ZAP.
- **VM 2 (juiceshop-server):** 192.168.100.20. Aloja o contentor Docker `bkimminich/juice-shop` a expor a aplicação na porta 3000.
- **VM 3 (waf-server):** 192.168.100.30. Servidor Apache com ModSecurity e OWASP CRS v3.3.5, utilizado na Fase 2 como proxy reverso para a VM 2.

Na Fase 1 o cliente comunica diretamente com 192.168.100.20:3000; na Fase 2 todo o tráfego passa primeiro pela VM 3 (192.168.100.30:80), que reencaminha para o JuiceShop após inspeção pelo ModSecurity.

1.2. Configuração do WAF (VM 3)

Para que o Apache funcionasse como proxy reverso e barreira de inspeção, foi criado o ficheiro `/etc/httpd/conf.d/juiceshop-waf.conf` com um `VirtualHost` a reencaminhar todos os pedidos para o JuiceShop e a forçar três cabeçalhos de segurança (`X-Frame-Options`, `X-Content-Type-Options` e `X-XSS-Protection`). O motor ModSecurity foi ativado em `/etc/httpd/conf.d/mod_security.conf` (`SecRuleEngine On` e `SecRequestBodyAccess On`) e foi adicionada uma regra customizada (`id:900001`) que força o uso do processador JSON sempre que o cabeçalho `Content-Type` é `application/json` — passo necessário porque o JuiceShop envia o login como JSON e, sem este detalhe, o CRS não inspecionava o corpo do pedido.

As regras de deteção foram ativadas a partir do OWASP Core Rule Set v3.3.5 instalado em `/etc/httpd/modsecurity.d/`. Foram ligadas em particular as regras `REQUEST-942-APPLICATION-ATTACK-SQLI.conf` (deteção de SQL Injection) e `REQUEST-949-BLOCKING-EVALUATION.conf` (avaliação final do anomaly score e bloqueio do pedido).

2. Testes WSTG ao OWASP Juice Shop – Fase 1

Os testes foram efetuados sobre `http://192.168.100.20:3000` com o OWASP ZAP 2.16.0 (add-ons Active scanner rules, Ajax Spider, Directory List, DOM XSS, Forced Browse, Fuzzer, Passive scanner rules e Reveal). Foi criada uma política de scan personalizada designada *FSI-Full-Scan* com *Threshold: Low* e *Strength: High*, de forma a maximizar a sensibilidade da deteção. Em 22/05/2026 foi ainda lançado um Active Scan autenticado com a conta `admin@juice-sh.op` através do método JSON-based authentication do ZAP.

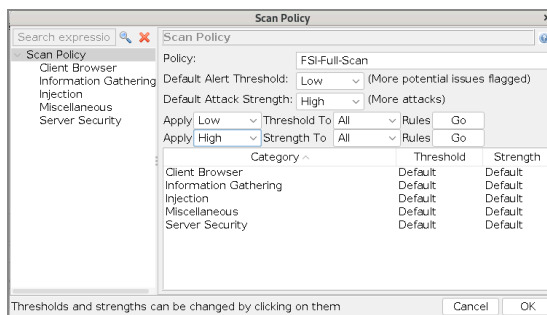


Fig. 1 – Política de scan *FSI-Full-Scan* utilizada no ZAP.

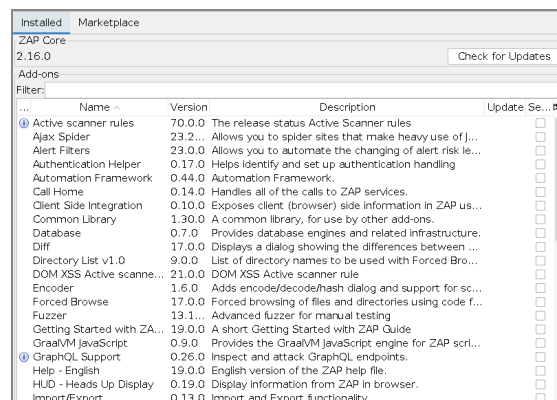


Fig. 2 – Add-ons do ZAP 2.16.0 instalados.

2.1. Recolha de Informação e Configurações Expostas

Logo nos primeiros pedidos verificou-se que a aplicação comunica em HTTP simples e revela detalhes da sua pilha tecnológica: cabeçalhos e mensagens de erro identificam Express 4.22.1, base de dados SQLite e front-end Angular, e em algumas respostas surgem caminhos internos do servidor e endereços IP privados. Ao nível de configuração,

o JuiceShop devolve `Access-Control-Allow-Origin: *` em todas as respostas (1803 instâncias), não define `Content-Security-Policy` (16 instâncias) e omite o cabeçalho `X-Frame-Options` (35 instâncias), o que o torna vulnerável a CORS abusivo, XSS e clickjacking. O scan autenticado revelou ainda quatro ficheiros escondidos no servidor.

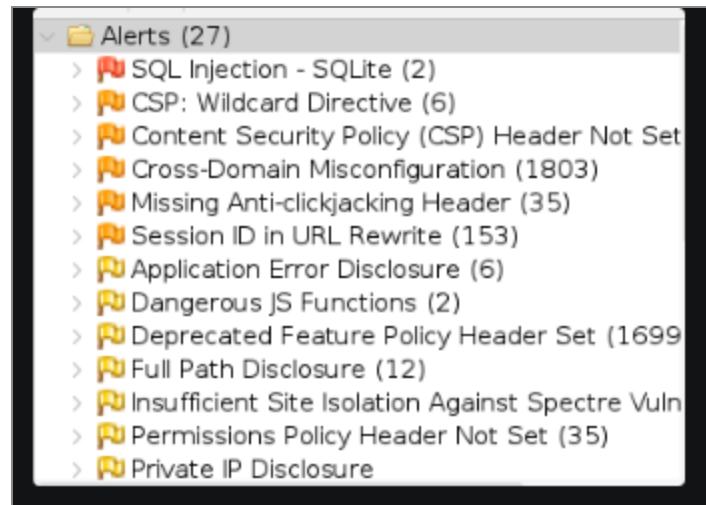


Fig. 3 – Painel de alertas do ZAP após o scan automático (27 tipos de alerta).

2.2. Gestão de Identidade e Autenticação

O painel `/#/administration` expõe a lista completa de utilizadores registados (10 emails, incluindo `admin@juice-sh.op`, `jim@juice-sh.op` e `bender@juice-sh.op`) assim que se entra com uma conta com privilégios. Para avaliar a robustez da autenticação foi utilizado o Fuzzer do ZAP contra `POST /rest/user/login`, com uma lista de passwords comuns. O teste contra a conta inexistente `test@test.com` devolveu sempre 401 Unauthorized; o teste contra `admin@juice-sh.op` devolveu 200 OK exatamente no payload `admin123` (resposta com 799 bytes contendo o token de sessão), o que confirma que a conta de administração tem uma password trivial.

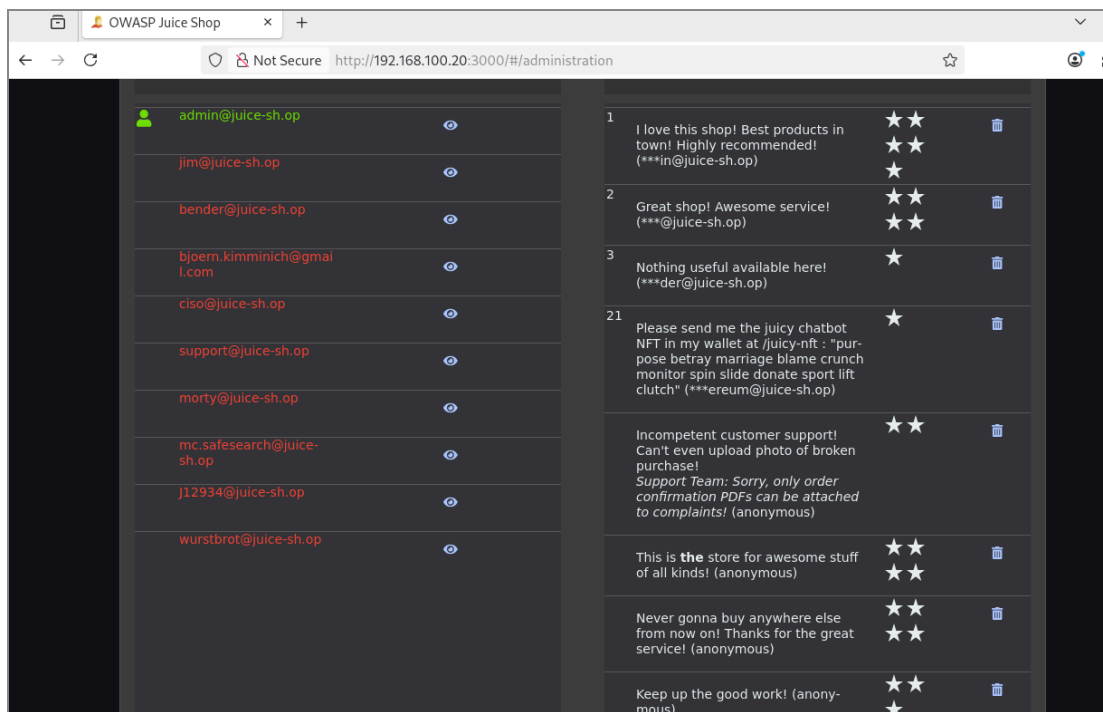


Fig. 4 – Painel /#/administration com a lista de utilizadores.

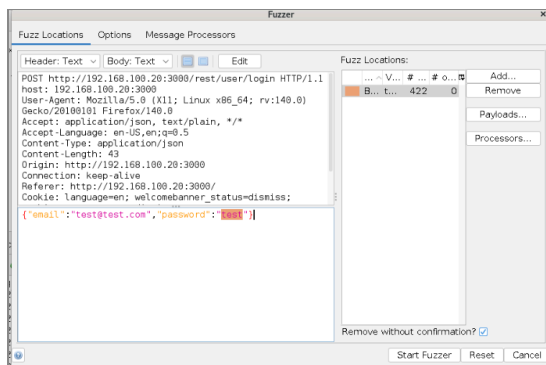


Fig. 5 – Configuração do Fuzzer para test@test.com.

Task ID	Message Type	Code	Reason	RTT	Size Resp. Header	Size Resp. Body	Highest Alert	State	Payloads
0	Original	401	Unauthorized	46 ms	387 bytes	26 bytes	Medium		admin
1	Fuzzed	401	Unauthorized	36 ms	387 bytes	26 bytes		Reflected	password1
2	Fuzzed	401	Unauthorized	41 ms	387 bytes	26 bytes			123456
3	Fuzzed	401	Unauthorized	25 ms	387 bytes	26 bytes			admin123
4	Fuzzed	401	Unauthorized	26 ms	387 bytes	26 bytes			password123
5	Fuzzed	401	Unauthorized	26 ms	387 bytes	26 bytes			admin@juice-sh.op
6	Fuzzed	401	Unauthorized	24 ms	387 bytes	26 bytes			admin@juice-sh.op
7	Fuzzed	401	Unauthorized	27 ms	387 bytes	26 bytes			admin@juice-sh.op

Fig. 6 – Resultados: todos 401 Unauthorized.

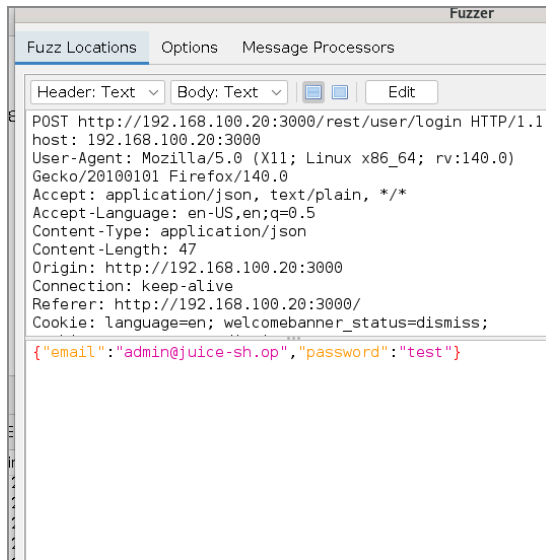


Fig. 7 – Fuzzer aplicado a *admin@juice-sh.op*.

0 Original	401 Unauthorized	22 ms	387 bytes	26 bytes	Medium	admin123
1 Fuzzed	200 OK	100 ms	386 bytes	799 bytes	Reflected	password
2 Fuzzed	401 Unauthorized	151 ms	387 bytes	26 bytes		admin
3 Fuzzed	401 Unauthorized	156 ms	387 bytes	26 bytes		123456
4 Fuzzed	401 Unauthorized	99 ms	387 bytes	26 bytes		password123
5 Fuzzed	401 Unauthorized	43 ms	387 bytes	26 bytes		P@ssw0rd
6 Fuzzed	401 Unauthorized	56 ms	387 bytes	26 bytes		admin@juic...
7 Fuzzed	401 Unauthorized	47 ms	387 bytes	26 bytes		

Fig. 8 – Linha com *admin123*: 200 OK (799 bytes).

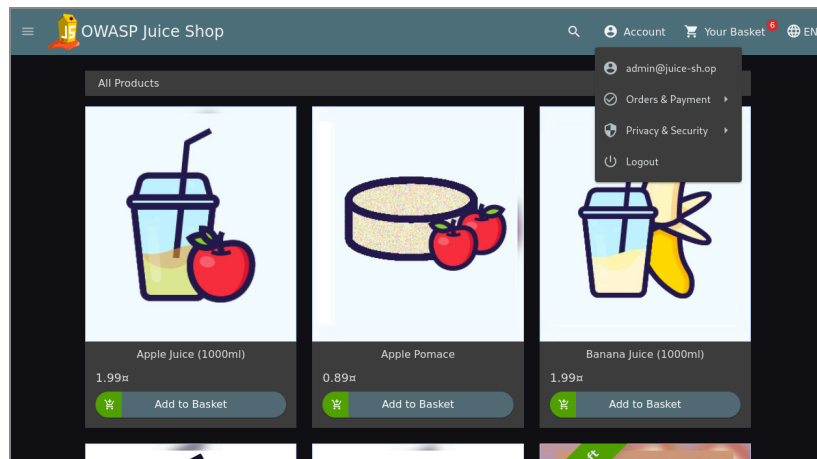


Fig. 9 – Login bem-sucedido com *admin@juice-sh.op/admin123*.

2.3. Autorização, Sessão e Validação de Input

Confirmada a posse de uma conta de administrador, o painel `/#/administration` permite consultar todos os utilizadores e respetivos comentários e até apagá-los, evidenciando uma falha grave de controlo de acesso. As baskets de utilizadores estão acessíveis em `/rest/basket/<id>` com identificadores sequenciais, o que abre porta a IDOR. Já a nível de sessão, observou-se que o token aparece em URLs (153 instâncias), o que o expõe em logs, histórico e Referer.

A vulnerabilidade mais grave está, no entanto, na validação de input do endpoint de login. O payload `' OR 1=1--` colocado no campo *email* obtém uma resposta 200 OK e o respetivo token de administrador, confirmando uma *SQL Injection* que permite contornar completamente a autenticação. O ZAP detetou ainda duas instâncias de uso de funções JavaScript perigosas (`eval()`, `innerHTML`) que podem permitir DOM XSS.

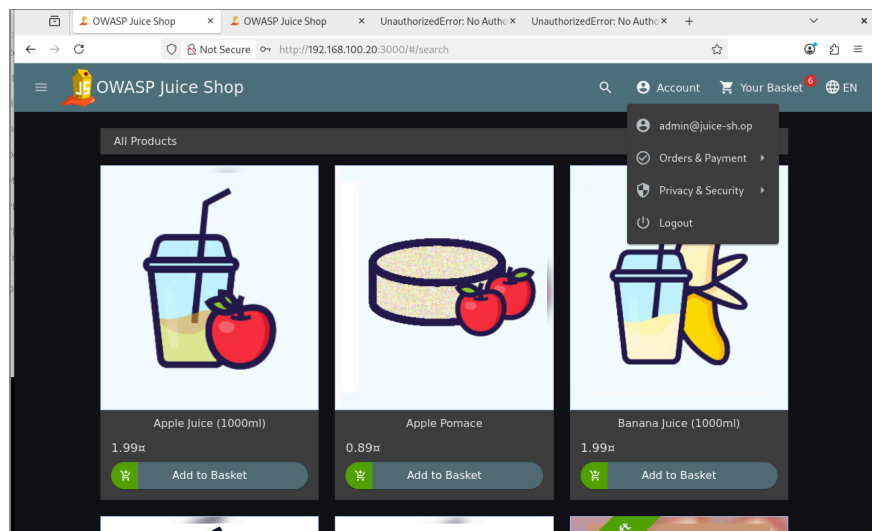


Fig. 10 – Sessão autenticada como administrador com acesso ao menu.

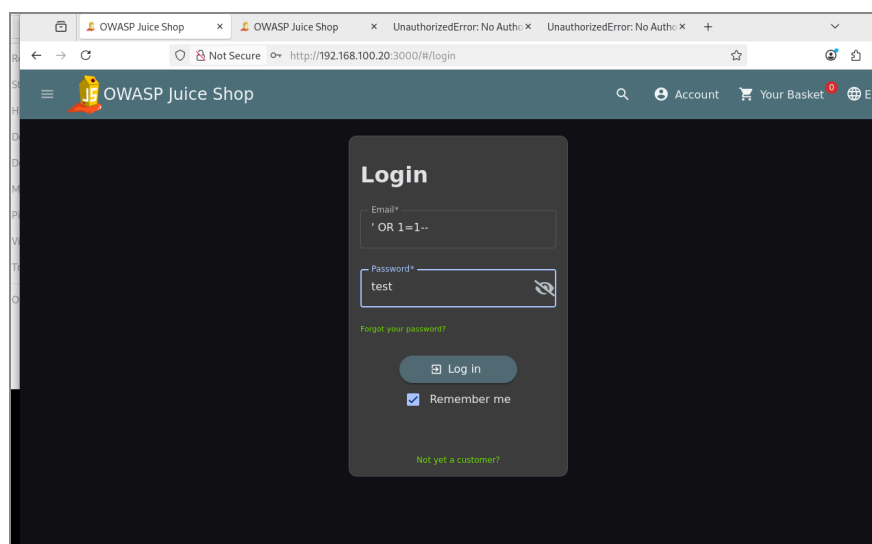


Fig. 11 – Formulário de login com payload ' OR 1=1 -- bem-sucedido.

2.4. Resumo das Vulnerabilidades Encontradas

Ao todo identificámos 18 vulnerabilidades em 9 das 11 categorias do WSTG (WSTG-CRYP e WSTG-BUSL ficam fora do âmbito, dado que a comunicação é em HTTP simples e a lógica de negócio não é exigida). As três vulnerabilidades mais graves — todas críticas — são o SQL Injection no login, a password trivial do administrador e o painel de administração aberto a qualquer conta com permissões. Seguem-se vários problemas de configuração (CORS wildcard, CSP ausente, falta de X-Frame-Options, X-Content-Type-Options e cabeçalhos de Site Isolation), exposição de tecnologia e caminhos internos nas mensagens de erro e o token de sessão em URL.

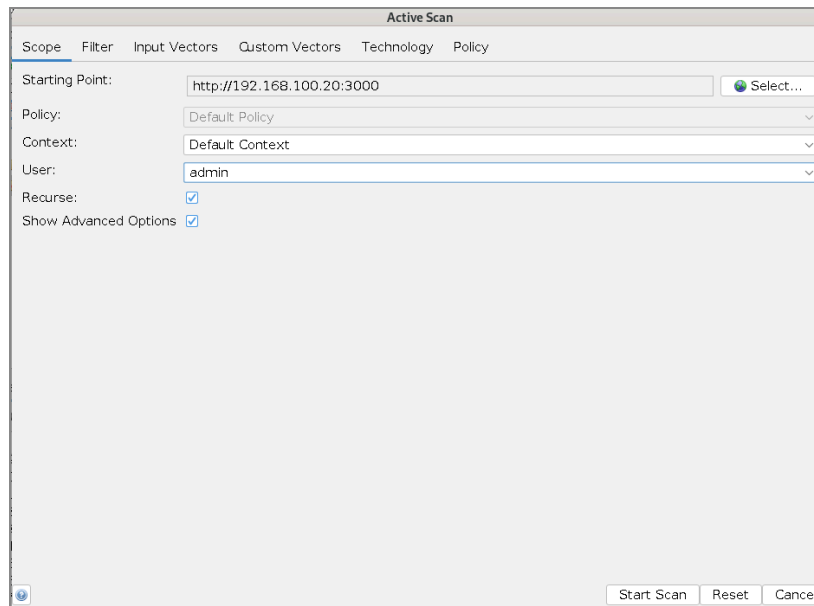


Fig. 12 – Configuração do Active Scan autenticado no ZAP.

#	Vulnerabilidade	WSTG	Severidade
1	SQL Injection no login (' OR 1=1--)	WSTG-INPV-05	Crítico
2	Password fraca do administrador (admin123)	WSTG-ATHN-07	Crítico
3	Painel de administração exposto	WSTG-ATHZ-02	Crítico
4	CORS wildcard (Access-Control-Allow-Origin: *)	WSTG-CONF-05	Alto
5	Enumeração de utilizadores	WSTG-IDNT-01	Alto
6	IDOR em /rest/basket/<id>	WSTG-ATHZ-01	Alto
7	Application + Full Path Disclosure	WSTG-ERRH-01	Médio
8	CSP ausente ou wildcard	WSTG-CONF-12	Médio
9	Falta de X-Frame-Options	WSTG-CONF	Médio
10	Session ID em URL	WSTG-SESS-04	Médio
11	DOM XSS – funções JS perigosas	WSTG-INPV-01	Médio
12	Private IP Disclosure	WSTG-INFO	Médio
13	Hidden File Found (autenticado)	WSTG-CONF-04	Médio

14	Timestamp Disclosure (Unix)	WSTG-INFO	Baixo
15	Falta de X-Content-Type-Options	WSTG-CONF	Baixo
16	Falta de Site Isolation / Permissions-Policy	WSTG-CLNT	Baixo
17	Suspicious Comments no código-fonte	WSTG-INFO	Informativo
18	Divulgação de Express/SQLite	WSTG-INFO	Informativo

3. Testes ao WAF e Comparação – Fase 2

Repetiram-se na Fase 2 os mesmos testes da Fase 1, desta vez sobre `http://192.168.100.30`. O Active Scan do ZAP foi corrido a 22/05/2026 às 17:07 e o relatório completo foi gravado em *reportAttackFase2.html*.

3.1. Bloqueio do SQL Injection

O resultado mais relevante é o bloqueio efetivo do SQL Injection de login: o mesmo pedido que na Fase 1 devolvia 200 OK passa agora a devolver **403 Forbidden**. A análise do `modsec_audit.log` mostra que o pedido é classificado pelas regras da família 942 (SQL Injection), com *SQLI anomaly score* = 8, ultrapassando o limiar definido pela 949 e ativando o bloqueio. Foi necessário ajustar o processamento JSON do ModSecurity (regra `id:900001`) porque o JuiceShop envia o login como `application/json` e, sem essa configuração, o CRS estava a ignorar o corpo do pedido.

```
# /etc/httpd/conf.d/juiceshop-waf.conf
<VirtualHost *:80>
    ProxyPass          / http://192.168.100.20:3000/
    ProxyPassReverse   / http://192.168.100.20:3000/
    Header always set  X-Frame-Options "SAMEORIGIN"
    Header always set  X-Content-Type-Options "nosniff"
    Header always set  X-XSS-Protection "1; mode=block"
</VirtualHost>
```

3.2. Limites do WAF

O WAF não substitui correções na aplicação. O fuzz attack ao endpoint de login continua a funcionar — o ModSecurity não tem rate-limiting no CRS base, pelo que o payload `admin123` volta a devolver 200 OK e a entregar o token de admin. Também não há nada que o WAF possa fazer contra a enumeração de utilizadores ou o IDOR: o JuiceShop é uma SPA Angular, os identificadores estão no fragmento `#` da URL e por isso não chegam ao servidor. Os cabeçalhos enviados pela aplicação (`Access-Control-Allow-Origin: *`, falta de CSP, token na URL) também passam intactos, porque o ModSecurity inspeciona pedidos mas não filtra respostas.

Ao mesmo tempo, o WAF introduziu dois alertas novos que o ZAP detetou. A diretiva `Header always set X-Frame-Options` aplica-se a respostas que o próprio JuiceShop já trazia com o mesmo cabeçalho, resultando em 880 instâncias de *Multiple X-Frame-Options*. E o Apache passa a anunciar a sua própria versão em `Server:`, gerando 24 637 instâncias de divulgação de versão. Ambos são corrigíveis com `Header always unset` antes do `Header set` e com `ServerTokens Prod/ServerSignature Off` no `httpd.conf`.

3.3. Comparação Fase 1 vs. Fase 2

Vulnerabilidade	Fase 1	Fase 2 (via WAF)	Resultado
SQL Injection no login	200 OK (admin)	403 Forbidden	Bloqueado
SQL Injection noutros endpoints	2 inst.	5 inst.	Parcial
Password fraca (admin123)	200 OK	200 OK	Não bloqueado
Painel /#/administration	Acessível	Acessível	Não bloqueado
CORS wildcard	980 inst.	980 inst.	Não bloqueado
CSP ausente	Presente	16 339 inst.	Não bloqueado
Falta de X-Frame-Options	35 inst.	Adicionado (com duplicado)	Mitigado / efeito lateral
Falta de X-Content-Type-Options	39 inst.	Adicionado	Mitigado
Session ID em URL	153 inst.	15 942 inst.	Não bloqueado
Application Error Disclosure	6 inst.	7 inst.	Não bloqueado
Hidden File Found	4 inst.	8 inst.	Não bloqueado
<i>Multiple X-Frame-Options</i> (novo)	—	880 inst.	Introduzido pelo WAF
<i>Server Version Leakage</i> (Apache)	—	24 637 inst.	Introduzido pelo WAF

4. Conclusão

Na Fase 1 demonstrou-se que o OWASP Juice Shop é, como esperado, uma aplicação proposadamente vulnerável: foram encontradas 18 vulnerabilidades, três das quais críticas (SQL Injection no login, password trivial do administrador e painel de administração exposto). A combinação do ZAP (scan automático, scan autenticado e Fuzzer) com testes manuais foi suficiente para cobrir 9 das 11 categorias do WSTG.

Na Fase 2 confirmou-se a utilidade e ao mesmo tempo os limites de um WAF. O Apache + ModSecurity + OWASP CRS v3.3.5 bloqueia eficazmente ataques de injeção

HTTP (o SQL Injection de login passou a devolver 403) e permite acrescentar facilmente cabeçalhos de segurança que a aplicação não envia. Em contrapartida, não consegue resolver problemas de lógica aplicacional (passwords fracas, falhas de autorização, IDOR em SPAs Angular) e pode até introduzir novos alertas se a sua configuração não for cuidada (cabeçalhos duplicados, divulgação da versão do Apache). Um WAF é uma camada de defesa relevante, mas não substitui boas práticas de desenvolvimento seguro na própria aplicação.